

Enumeração de Ciclos em Grafos: Permutação e Caminhamento

Diego Polanski de Freitas Vieira

polanskidiego@gmail.com

Vitor Dias de Britto Militão

vdiasmilitao@gmail.com

Vitor Alexandre Moreira Amaral

vitoramamaral@gmail.com

Mateus Nunes Bello

nunesbellom@gmail.com

Pedro Henrique Bellone Souza e Silva

pedrohbsyt@gmail.com

Nalberth Henrique Vieira Pinto

nalberthvieira@gmail.com

✉ Pontifícia Universidade Católica de Minas Gerais, R. Dom José Gaspar, 500 - Coração Eucarístico, Belo Horizonte - MG, 30535-901

Abstract. Algoritmos baseados em grafos são amplamente utilizados em diversas áreas para a solução de problemas computacionais. Neste trabalho, abordamos o problema da enumeração de todos os ciclos em um grafo simples não direcionado. Para isso, implementamos e comparamos duas abordagens distintas: (i) um método baseado na permutação dos vértices e (ii) um método baseado em caminhamento no grafo. A eficiência de ambas as abordagens foi analisada considerando o desempenho à medida que o tamanho do grafo aumenta. Além disso, discutimos a escolha da representação do grafo e avaliamos a aplicabilidade das implementações para grafos direcionados. Os resultados experimentais evidenciam as diferenças entre as abordagens, fornecendo insights sobre seu uso prático.

Keywords: Grafos, Ciclos, BFS, Permutação

1 Introdução

A Teoria dos Grafos é uma área fundamental da Matemática e da Ciência da Computação, com aplicações diretas em redes de computadores, inteligência artificial, logística, biologia computacional e análise de redes sociais. Um dos problemas clássicos dessa área é a identificação e enumeração de ciclos em grafos. Em um grafo simples não direcionado, um ciclo é uma sequência fechada de vértices conectados por arestas, sem repetição de vértices intermediários. A detecção eficiente de ciclos é essencial em diversas aplicações, como análise de dependências computacionais, interações químicas e circuitos elétricos.

Neste trabalho, implementamos e comparamos duas abordagens para a enumeração de ciclos: (i) um método baseado na permutação dos vértices e (ii) um método baseado no caminhamento no grafo. O primeiro gera diferentes ordenações de vértices para identificar ciclos válidos, enquanto o segundo percorre a estrutura do grafo utilizando técnicas de busca. Cada abordagem apresenta vantagens e desafios, especialmente em termos de escalabilidade e eficiência computacional.

Além da implementação, realizamos uma análise comparativa detalhada, avaliando o impacto do aumento de vértices e arestas no desempenho de cada método. Também discutimos a representação do grafo adotada e a possibilidade de adaptação das soluções para grafos direcionados.

Nosso principal objetivo é compreender quais técnicas são mais eficientes na enumeração de ciclos e em quais contextos se aplicam melhor. A partir dos resultados experimentais, buscamos identificar limitações e potenciais otimizações, contribuindo para um melhor entendimento do prob-

lema e suas aplicações na Computação.

Usaremos como base para comparação entre os métodos o grafo a seguir:

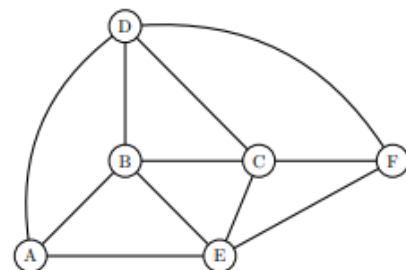


Figura 1: Exemplo de grafo com 6 vértices e 11 arestas.

2 Detecção de Ciclos por Permutação

A classe `DetectaCiclosPermutacao` tem como objetivo identificar ciclos em um grafo não direcionado por meio de permutações dos vértices do grafo. O principal objetivo dessa classe é gerar todas as permutações possíveis dos vértices e verificar se existe um caminho cíclico entre os vértices de uma permutação específica. O algoritmo realiza essa detecção em duas funções principais: `PossuiCaminhoEntreVertice` e `gerarPermutacoes`.

2.1 Função `gerarPermutacoes`

Esta função é responsável por gerar todas as permutações possíveis de vértices no grafo e verificar se algum ciclo é detectado. Ela recebe os seguintes parâmetros:

- **totalVertices**: O número total de vértices no grafo.
- **tamanhoGrupo**: O tamanho do ciclo que se deseja detectar (geralmente, maior que 2 para formar ciclos válidos).
- **tamanhoAtual**: O número de vértices já selecionados para a permutação atual.
- **grupoAtual**: Um vetor de inteiros que armazena os vértices da permutação atual.
- **totalPermutacoes**: Um vetor que armazenará todas as permutações geradas.

A função realiza o seguinte processo:

Algorithm 1 Geração de Permutações de Vértices

Input: *totalVertices* (número total de vértices), *tamanhoGrupo* (tamanho do grupo), *tamanhoAtual* (tamanho atual da permutação), *grupoAtual* (permutação parcial), *totalPermutacoes* (matriz de saída)

Output: Lista de todas as permutações possíveis

```

1 if tamanhoAtual = tamanhoGrupo then Adicionar grupoAtual a totalPermutacoes return end if
2 for i ← 0 até totalVertices - 1 do if i ∉ grupoAtual then Adicionar i a grupoAtual GerarPermutacoes(totalVertices, tamanhoGrupo, tamanhoAtual + 1, grupoAtual, totalPermutacoes) Remover último elemento de grupoAtual end if
3 end for

```

2.2 Função PossuiCaminhoEntreVertice

Esta função tem como objetivo verificar se existe um caminho cíclico entre os vértices de uma permutação dada, levando em consideração a matriz de adjacência do grafo. Ela recebe dois parâmetros:

- **aresta**: Um vetor de inteiros que representa uma permutação dos vértices do grafo. A função verifica se existe um caminho entre os vértices consecutivos dessa permutação.
- **matrizAdj**: A matriz de adjacência que representa o grafo. Um valor de 1 na posição *matrizAdj*[i][j] indica que existe uma aresta entre os vértices i e j.

A função realiza o seguinte processo:

Algorithm 2 Verifica se um conjunto de vértices forma um ciclo

Input: *aresta*: vetor de inteiros representando os vértices percorridos
matrizAdj: matriz de adjacência do grafo

Output: **true** se os vértices formam um ciclo válido, caso contrário **false**

```

4 if |aresta| < 3 then
5   return false
6 end if
7 for i ← 1 to |aresta| - 1 do
8   if matrizAdj[aresta[i - 1]][aresta[i]] ≠ 1 then
9     return false
10  end if
11 end for
12 return (matrizAdj[aresta[|aresta| - 1]][aresta[0]] = 1)

```

2.3 Função encontrarCiclos

Esta função tem como objetivo encontrar ciclos em um grafo, considerando ciclos de diferentes tamanhos, a partir de 3 vértices até o número total de vértices do grafo. O funcionamento principal dessa função é iterar sobre os possíveis tamanhos de ciclo, gerar todas as permutações possíveis de vértices para cada tamanho de ciclo e verificar se essas permutações formam ciclos válidos. Para isso, ela utiliza as funções *gerarPermutacoes* e *PossuiCaminhoEntreVertice*.

A função recebe um único parâmetro:

- **g**: O grafo no qual os ciclos serão encontrados. O grafo é representado por uma classe *Grafo*, que contém a matriz de adjacência do grafo.

O funcionamento da função é da seguinte forma:

Algorithm 3 Inicializa os Vetores e Enumera os Ciclos no Grafo

Input: *g*: O grafo contendo a matriz de adjacência

Output: Impressão do total de ciclos encontrados para cada tamanho de ciclo e o total geral de ciclos

Data: *matrizAdj*: Matriz de adjacência do grafo, *totalVertices*: número de vértices no grafo

```

13 for tamanhoGrupo ← 3 to totalVertices do
14   GerarPermutacoes(totalVertices, tamanhoGrupo, 0, [], grupos)
15   ciclosPorTamanho ← 0 foreach grupo em grupos do
16     if PossuiCaminhoEntreVertice(grupo, matrizAdj) then
17       ciclosPorTamanho ← ciclosPorTamanho + 1
18     end if
19   end foreach
20   ciclosAjustados ←  $\frac{\text{ciclosPorTamanho}}{\text{tamanhoGrupo} \times 2}$  totalCiclos
21   ← totalCiclos + ciclosAjustados Imprimir "Total de ciclos encontrados para tamanho tamanhoGrupo: ciclosAjustados;"
22 end for
23 Imprimir "Total geral de ciclos encontrados: totalCiclos;"

```

2.4 Eficiência e Garantia de Ciclos Únicos

A abordagem de permutação garante que todos os caminhos possíveis sejam explorados, porém, a geração de permutações pode ser computacionalmente cara, principalmente para grafos grandes. A função *gerarPermutacoes* gera todas as permutações possíveis dos vértices, o que leva a uma complexidade de $O(V!)$, onde V é o número de vértices no grafo.

A função *PossuiCaminhoEntreVertice* verifica se a permutação formada constitui um ciclo válido, o que tem uma complexidade de $O(V)$, já que precisa percorrer os vértices da permutação. Assim, a complexidade geral do algoritmo pode ser estimada como $O(V! \cdot V)$, o que pode ser muito alto para grafos grandes.

O algoritmo garante a não repetição de ciclos, uma vez executa uma verificação dividindo por 2.

2.4.1 Complexidade do Algoritmo

A complexidade do algoritmo é dominada pela geração das permutações, que é $O(V!)$, multiplicada pela verificação do

caminho cíclico, que é $O(V)$ para cada permutação. Portanto, a complexidade total do algoritmo é $O(V! \cdot V)$. Esse algoritmo é mais adequado para grafos pequenos ou médios, onde o número de vértices não é muito grande, permitindo que a geração de permutações seja computacionalmente viável. Para grafos maiores, esse método pode ser ineficiente, e abordagens alternativas, como busca em profundidade ou algoritmos de detecção de ciclos mais especializados, podem ser mais apropriadas.

3 Detecção de Ciclos por Caminhamento

A classe `DetectaCiclos` tem como objetivo identificar ciclos em um grafo não direcionado utilizando a técnica de Busca em Profundidade (DFS). O principal objetivo desse código é explorar os vértices do grafo de forma a encontrar ciclos, evitando ciclos repetidos e garantindo que a detecção seja eficiente. A implementação consiste em duas funções principais: `encontrarCiclos` e `buscarCiclos`.

3.1 Função `encontrarCiclos`

Esta função é chamada para iniciar a busca por ciclos no grafo. Ela percorre todos os vértices do grafo, chamando a função `buscarCiclos` para cada vértice. O primeiro parâmetro da função `buscarCiclos` é o vértice atual, e o segundo parâmetro é o vértice de início, garantindo que a busca será realizada a partir de um vértice específico. A função mantém um conjunto de ciclos únicos, representados como strings, para garantir que ciclos repetidos não sejam contados várias vezes. O cálculo do número total de ciclos é realizado dividindo o tamanho do conjunto por 2, uma vez que cada ciclo é contado em ambas as direções, o que resulta em uma duplicação dos ciclos.

Algorithm 4 Encontrar Ciclos no Grafo

Input: g : O grafo contendo a matriz de adjacência

Output: Impressão do total de ciclos encontrados

Data: `matrizAdj`: Matriz de adjacência do grafo, n : número de vértices

```

22 for  $i \leftarrow 0$  to  $n - 1$  do
23   visitados  $\leftarrow$  vetor de tamanho  $n$  com false
24   BuscarCiclos( $i, i, \{\}, \text{visitados}, i, \text{matrizAdj}$ )
25 end for
26 Imprimir "Total de ciclos encontrados:  $\frac{\text{ciclosUnicos.size}()}{2}$ "

```

3.2 Função `buscarCiclos`

Esta função é a implementação recursiva da Busca em Profundidade. Ela recebe os seguintes parâmetros:

- **atual**: Representa o vértice que está sendo processado no momento na busca.
- **inicio**: Representa o vértice onde o ciclo começou, usado para verificar se encontramos um ciclo completo.

- **caminho**: Uma lista contendo os vértices que foram visitados até o momento no caminho atual. A lista ajuda a reconstruir o ciclo quando ele é detectado.
- **visitados**: Um conjunto que mantém o registro dos vértices já visitados, evitando que o algoritmo entre em ciclos infinitos.
- **menorInicio**: O menor vértice que foi encontrado até o momento no ciclo, utilizado para padronizar a rotação do ciclo e evitar contagens duplicadas.

O funcionamento da função é o seguinte:

Algorithm 5 Buscar Ciclos no Grafo

Input: `atual`: Vértice atualmente sendo visitado

Input: `inicio`: Vértice de início do ciclo

Input: `caminho`: Vetor contendo o caminho atual percorrido

Input: `visitados`: Vetor de vértices visitados

Input: `menorInicio`: Menor vértice no ciclo até o momento

Input: `matrizAdj`: Matriz de adjacência do grafo

Output: Ciclos encontrados, armazenados em `ciclosUnicos`

```

26 Adicionar atual a caminho   Marcar atual como visitado   for
    vizinho  $\leftarrow 0$  to matrizAdj.size() - 1 do
27   if matrizAdj[atual][vizinho] == 1 then
28     if vizinho == inicio and |caminho| > 2 then
29       Criar ciclo como uma cópia de caminho   Rotacionar
        ciclo para que o menor vértice esteja no início   Converter
        ciclo em uma string cicloStr   Adicionar cicloStr a
        ciclosUnicos
30     end if
31     else if vizinho != inicio and !visitados[vizinho] and vizinho >
        menorInicio then
32       BuscarCiclos(vizinho, inicio, caminho, visitados,
        menorInicio, matrizAdj)
33     end if
34   end if
35 end for

```

3.2.1 Detecção de Ciclo

Caso o vértice adjacente seja o mesmo que o vértice de início (`inicio`) e o tamanho do caminho seja maior que 2 (indicando que o ciclo possui ao menos três vértices), um ciclo foi detectado. O ciclo é extraído da lista `caminho` e convertido em um vetor de inteiros. Para garantir que o ciclo seja armazenado de forma única, ele é rotacionado para que o menor vértice no ciclo esteja sempre na posição inicial. Isso padroniza a forma como os ciclos são registrados, evitando a contagem de ciclos equivalentes em diferentes orientações. O ciclo é convertido em uma string e inserido no conjunto `ciclosUnicos`, garantindo que ciclos repetidos não sejam contados.

3.2.2 Continuação da Busca

Se o vértice adjacente ainda não foi visitado e o número do vértice adjacente for maior do que o `menorInicio` (o que ajuda a evitar a exploração de ciclos em direção contrária ao ciclo detectado), a busca é chamada recursivamente para o vértice adjacente.

3.3 Eficiência e Garantia de Ciclos Únicos

O uso do conjunto `ciclosUnicos` é uma estratégia eficaz para garantir que ciclos não sejam duplicados, uma vez que o mesmo ciclo pode ser encontrado em diferentes ordens durante a busca. Além disso, a padronização do ciclo pela rotação para o menor vértice é crucial para evitar a contagem de ciclos em diferentes orientações.

A função de busca recursiva em profundidade permite a exploração completa do grafo e a detecção de ciclos, enquanto o uso do conjunto de vértices visitados impede a exploração de ciclos triviais e facilita a identificação de ciclos reais. Esse algoritmo é eficiente para grafos de tamanho médio, mas pode se tornar menos eficiente à medida que o grafo cresce, devido à necessidade de explorar múltiplos caminhos de maneira recursiva.

3.3.1 Complexidade do Algoritmo

A complexidade do algoritmo depende de vários fatores, incluindo o número de vértices V e o número de arestas E no grafo. O algoritmo realiza uma busca em profundidade, o que, em termos de complexidade, é $O(V + E)$ para um grafo não ponderado, já que cada vértice e cada aresta será visitado.

Além disso, o algoritmo precisa verificar todos os ciclos possíveis. A geração de ciclos não triviais é uma operação adicional, e, no pior cenário, o número de ciclos pode ser exponencial em relação ao número de vértices, $O(2^V)$, especialmente para grafos densos ou completos. No entanto, o uso do conjunto `ciclosUnicos` e a padronização pela rotação ajudam a reduzir a redundância e garantir que apenas ciclos únicos sejam contados, o que melhora a eficiência na prática, mas não altera a complexidade assintótica geral.

Portanto, a complexidade total do algoritmo, levando em consideração a detecção de ciclos e as operações de verificação, pode ser considerada como $O(V + E + 2^V)$ no pior caso.

4 Conclusão

4.1 Análise Comparativa das Abordagens

A análise comparativa entre as abordagens de detecção de ciclos por permutação e por caminhamento revelou diferenças significativas em termos de desempenho. O método baseado em permutação, embora garanta a exploração completa dos ciclos possíveis, apresenta uma complexidade exponencial $O(V! \cdot V)$, tornando-o inviável para grafos grandes. Em contrapartida, o método baseado em caminhamento utiliza a busca em profundidade (DFS), reduzindo a complexidade para $O(V + E + 2^V)$, o que é mais eficiente para a maioria dos casos práticos, especialmente para grafos esparsos.

4.2 Impacto do Tamanho do Grafo

O impacto do crescimento do número de vértices e arestas é evidente na execução dos algoritmos. A abordagem de permutação sofre com o crescimento fatorial do número de vértices, tornando-se rapidamente impraticável. Já a abordagem baseada em caminhamento, apesar de ainda ter um crescimento exponencial no pior caso, consegue lidar melhor com o aumento da complexidade estrutural do grafo devido ao uso de estruturas como conjuntos de ciclos únicos para evitar contagens redundantes.

4.3 Limitações em Grafos Direcionados

A implementação apresentada não é capaz de lidar com grafos direcionados. O método baseado em permutação não considera a direção das arestas ao verificar a existência de ciclos, assumindo sempre uma estrutura não direcionada. Da mesma forma, o método baseado em caminhamento utiliza uma busca em profundidade que percorre as conexões sem levar em conta a direção das arestas, o que pode levar à detecção incorreta de ciclos em um grafo direcionado. Para adaptar esse código a grafos direcionados, seriam necessárias modificações significativas, incluindo ajustes na representação do grafo e na lógica de detecção de ciclos para garantir que apenas caminhos válidos dentro da orientação das arestas sejam considerados.

4.4 Considerações Finais

Os resultados experimentais mostraram que a detecção de ciclos baseada em caminhamento é a mais viável para grafos maiores devido à sua eficiência computacional superior em relação à abordagem de permutação. No entanto, para grafos pequenos, a abordagem de permutação pode ser útil para garantir a enumeração completa dos ciclos. A escolha do método mais adequado depende, portanto, do tamanho e da densidade do grafo, além das restrições computacionais disponíveis. Para estudos futuros, seria interessante investigar otimizações adicionais para ambas as abordagens e explorar heurísticas que possam reduzir a complexidade da enumeração de ciclos.

5 Atribuições e Responsabilidades

- **Documentação:** Diego Polanski e Vitor Militão
- **Caminhamento:** Mateus Nunes, Vitor Alexandre e Diego Polanski
- **Permutação:** Vitor Militão, Pedro Bellone e Nalberth Vieira